

Smart Crane Pathfinding System (智能起重机路径规划系统)

Status Prototype Python 3.8+ Vue.js Three.js

1. 项目概览 (Project Overview)

Smart Crane Pathfinding System 是一个专为工业天车 (Overhead Crane) 设计的智能调度与路径规划原型系统。该系统旨在解决复杂工业环境下的自动避障与最优路径解算问题。

系统采用了 **B/S 架构**，后端基于 Python 构建核心算法引擎，前端利用 WebGL 技术实现数字孪生级的可视化监控。

核心价值

- 动态环境适应**: 利用 D* Lite 算法处理移动障碍物，实现毫秒级路径重规划。
- 运动学优化**: 生成的路径符合起重机机械运动约束，通过平滑算法消除“折线”运动。
- 实时监控**: 提供 2D 拓扑与 3D 实景的双模态实时同步视图。

2. 关键特性 (Key Features)

智能寻路引擎

- A (A-Star)***: 用于静态地图初始化时的全局最优路径搜索。
- D Lite (Dynamic A)****: 核心亮点。在障碍物发生位移时，仅重算受影响的节点 (Incremental Search)，而非全量重算，极大提升了动态响应速度。
- 持久化规划器: TrajectoryPlanner** 在服务层常驻内存，维护搜索树状态，确保增量算法生效。

轨迹后处理 (Post-Processing)

- 关键点提取 (Greedy)**: 去除冗余路径点，提取直线段关键转折点。
- 贝塞尔平滑 (Bezier)**: 将折线路径转化为平滑曲线，模拟起重机大车与小车的协同运动轨迹。

可视化交互

- 3D 数字孪生**: 基于 Three.js 构建，支持自由视角旋转、缩放。
- 实时通信**: 基于 Socket.IO 实现前后端双向低延迟通信 (位置更新、地图变更、日志推送)。

3. 系统架构 (System Architecture)

本项目采用分层架构设计，确保关注点分离：

分层	模块	职责描述
表现层	static/js/	Vue.js + Three.js. 负责用户交互、3D 场景渲染 (Canvas3d)、2D 网格渲染 (Canvas2d) 及状态同步。

分层	模块	职责描述
接口层	app.py	Flask + Socket.IO。处理 HTTP 请求与 WebSocket 事件，路由分发。
业务层	core/crane_service.py	核心控制器。维护单例 CraneService，管理地图状态 (MapManager) 与规划器生命周期。
算法层	algorithms/	计算引擎。包含 AStar、DSLite 实现及 trajectory_planner.py 策略封装。
数据层	core/map_manager.py	内存数据。管理栅格地图数据结构，提供线程安全的读写访问 (Thread Lock) 。

4. 数据流向 (Data Pipeline)

场景：动态障碍物规避

- 事件触发:** 用户在前端 2D 视图拖动障碍物。
- 指令发送:** 前端通过 Socket 发送 `update_map` 事件，携带变化的坐标 (x, y) 和新状态。
- 状态更新:**
 - 后端 `app.py` 接收事件，调用 `CraneService.update_map()`。
 - `MapManager` 更新底层栅格数据（加锁）。
 - 关键步骤:** `CraneService` 通知持久化的 `self.planner` 实例，调用 `dslite.update_cell()` 更新受影响节点的 `rhs` 值。
- 重规划:** `CraneService` 触发 `find_path`，算法基于现有搜索树快速修补路径。
- 反馈渲染:** 新轨迹点通过 WebSocket 推送至前端，驱动 3D 起重机模型按新路径运动。

5. 快速开始 (Getting Started)

前置要求

- Python 3.8 或更高版本
- 现代浏览器 (Chrome/Edge/Firefox) 支持 WebGL

安装与运行

1. 克隆项目

```
git clone <repository_url>
cd smart-crane-astar-pathfind-python
```

2. 安装依赖

```
pip install -r requirements.txt
```

3. 启动服务

```
python app.py
```

4. 访问系统 打开浏览器访问: <http://127.0.0.1:5000>

6. 项目结构 (Project Structure)

```
.
├── algorithms/                # 核心算法模块
│   ├── astar.py              # A* 算法实现
│   ├── dslite.py             # D* Lite 算法实现 (增量更新)
│   ├── trajectory_planner.py # 轨迹规划器 (策略上下文)
│   └── post_processing/      # 路径平滑与优化
├── core/                     # 业务逻辑核心
│   ├── crane_service.py     # 起重机服务 (状态机、持久化实例)
│   ├── map_manager.py       # 地图数据管理 (线程安全)
│   └── config.py            # 全局配置
├── static/                   # 前端资源
│   ├── js/
│   │   ├── components/     # Vue 组件 (Canvas2d, Canvas3d, LogPanel)
│   │   └── app.js           # Vue 入口逻辑
│   └── vendor/              # 第三方库 (Three.js, Socket.IO, Tailwind)
├── templates/                # HTML 模板
└── app.py                    # Flask 应用入口
```

7. 未来规划 (Roadmap)

短期目标

- ❑ **持久化存储:** 引入 SQLite 数据库, 存储历史运行轨迹和报警日志。
- ❑ **多机协同:** 扩展 `CraneService` 以支持多台起重机在同一地图下的防碰撞调度。

长期目标 (工业化)

- ❑ **Rust 重构:** 针对超大规模地图 (如 1000x1000+), 使用 Rust 重写 `algorithms/` 层并通过 PyO3 绑定, 以获得 10-50 倍的性能提升和内存安全性。
- ❑ **硬件接入:** 开发 PLC 适配层 (Modbus/OPC UA), 接入真实的西门子/三菱 PLC 控制器。